

UNIVERSITY OF
OXFORD

Large Language Models for Health and Social Science Research, June 2026

Text is an artefact of communication

- ▶ Text is not the goal itself; it is an artefact produced in service of communicative purposes:
to inform, coordinate, persuade, record, explain, or signal.
- ▶ Treating text as an artefact shifts the question:
What does the text say? → What does the text preserve, lose, or transform?
- ▶ This matters because NLP does not operate on communication directly. It operates on textual traces of communicative acts.

Starting point

NLP is computation over an already encoded communicative artefact.

Text becomes data when treated as a trace

- To reason formally about text, we need a framework for:

encoding, transmission, compression, loss.

- This framework lets us distinguish the phenomenon from its observable representation:

phenomenon $\longrightarrow$ textual trace $\longrightarrow$ computed representation.

Text as data

Text is data because it is an observable trace of a phenomenon we usually cannot observe directly.

- By the mid-twentieth century, long-distance communication had become an engineering problem at industrial scale:

telegraphy, telephony, radio, cable transmission, signal storage.

- At Bell Telephone Laboratories, the practical question was very concrete:

How can we transmit messages accurately through noisy physical media?

- The problem was not abstract philosophy. It was wires, signals, interference, storage, and reconstruction.

The engineering question

What structure must a communication system have in order to preserve a message across a noisy channel?

Shannon's abstraction

- ▶ Claude Shannon's 1948 paper, *A Mathematical Theory of Communication*, proposed a general formal answer:

treat messages as random variables generated by a source.

- ▶ This made communication mathematically tractable:

$$X \sim p(x), \quad x \in \mathcal{X}.$$

- ▶ The radical move was to separate the engineering problem of transmission from the philosophical problem of meaning:

information $\neq$ semantics.

Shannon's abstraction

A message can be studied through its probability structure: how uncertain it is, how it can be encoded, and how much of it survives transmission through a noisy channel.

Information begins with uncertainty

- ▶ Let X be a discrete random variable taking values $x \in \mathcal{X}$ according to a probability mass function $p(x)$.
- ▶ Shannon's first move is to measure the information associated with observing a particular outcome x :

$$h(x) = -\log_2 p(x).$$

- ▶ This quantity is usually called *self-information* or *surprisal*.
- ▶ The intuition is simple:

$$p(x) \downarrow \implies h(x) \uparrow.$$

- ▶ Rare events are more informative because they remove more uncertainty.

Unit

With logarithm base 2, information is measured in *bits*.

Entropy is expected information

- ▶ Self-information applies to one realised outcome. Entropy applies to the source as a whole.
- ▶ The entropy of X is the expected self-information:

$$H(X) = \mathbb{E}[h(X)] = - \sum_{x \in \mathcal{X}} p(x) \log_2 p(x).$$

- ▶ Entropy therefore measures the average uncertainty produced by a random variable before we observe its realisation.
- ▶ It depends only on the probability distribution over possible outcomes:

$$H(X) = H(p(x)).$$

- ▶ It does *not* measure meaning, truth, usefulness, or importance.

Important restriction

Shannon information is a property of probabilistic structure, not semantic content.

- ▶ Once we have more than one variable, we can ask how much observing one variable tells us about another.
- ▶ Conditional entropy measures residual uncertainty:

$$H(X | Y) = - \sum_{x,y} p(x, y) \log_2 p(x | y).$$

- ▶ Mutual information measures the reduction in uncertainty about X after observing Y :

$$I(X; Y) = H(X) - H(X | Y).$$

- ▶ Mutual information is symmetric:

$$I(X; Y) = I(Y; X).$$

For our purposes

Mutual information lets us ask how much an observable representation preserves about a source.

Communication as a chain of encodings

- ▶ Suppose Alice has a message M that she wants Bob to recover.
- ▶ The message is never transmitted directly. It moves through a sequence of representations:

$$M_{\text{mind}} \xrightarrow{f} M_{\text{speech}} \xrightarrow{g} M_{\text{text}} \xrightarrow{\text{channel}} M'_{\text{text}}.$$

- ▶ Bob then attempts to decode the received representation:

$$M'_{\text{text}} \xrightarrow{g^{-1}} \hat{M}_{\text{speech}} \xrightarrow{f^{-1}} \hat{M}_{\text{mind}}.$$

- ▶ The important point is that each arrow is a transformation of the currently available representation.

Communication

Communication is not the movement of meaning itself, but the movement and reconstruction of encoded representations.

Encoding is not always invertible

- ▶ Some transformations are approximately faithful:

$$f : M_{\text{mind}} \rightarrow M_{\text{speech}}.$$

- ▶ Others are intrinsically lossy:

$$g : M_{\text{speech}} \rightarrow M_{\text{text}}.$$

- ▶ Speech contains information that text often collapses or removes:

tone, emphasis, prosody, hesitation, gesture.

- ▶ Once collapsed, these distinctions are not available to downstream processing unless an additional channel provides them.

Loss

If an encoding is non-injective, different source states may produce the same observable representation.

Information can be preserved, but not invented

- ▶ Along the encoding chain, information about the original source can only be preserved or lost:

$$I(M_{\text{mind}}; M_{\text{speech}}) \geq I(M_{\text{mind}}; M_{\text{text}}) \geq I(M_{\text{mind}}; M'_{\text{text}}).$$

- ▶ Equivalently, uncertainty about the original source can only stay the same or increase:

$$H(M_{\text{mind}} | M_{\text{speech}}) \leq H(M_{\text{mind}} | M_{\text{text}}) \leq H(M_{\text{mind}} | M'_{\text{text}}).$$

- ▶ We can write the loss at each stage as:

$$\Delta_{f \rightarrow g} = I(M_{\text{mind}}; M_{\text{speech}}) - I(M_{\text{mind}}; M_{\text{text}}) \geq 0,$$

$$\Delta_{\text{channel}} = I(M_{\text{mind}}; M_{\text{text}}) - I(M_{\text{mind}}; M'_{\text{text}}) \geq 0.$$

Consequence

The received text may be useful, rich, and interpretable; but it cannot contain more information about the original source than survived the channel.

The Data Processing Inequality

- ▶ Suppose three random variables form a Markov chain:

$$X \longrightarrow Y \longrightarrow Z.$$

- ▶ This means that Z is produced by processing Y , not by observing X directly.
- ▶ The Data Processing Inequality states:

$$I(X; Z) \leq I(X; Y).$$

- ▶ In words:

processing data cannot increase information about its source.

Core intuition

A transformation can preserve information, compress it, reorganise it, or destroy it. It cannot create new information about the upstream source.

A canonical structure for text and data

- ▶ Let S denote the latent source we care about:

$S =$ belief, experience, diagnosis, attitude, event, parameter.

- ▶ Let O denote the observable encoding:

$O =$ text, transcript, survey response, clinical note, table.

- ▶ Let $T(O)$ denote any downstream representation:

$T(O) =$ tokens, embeddings, topics, summaries, predictions.

- ▶ The information structure is:

$$S \longrightarrow O \longrightarrow T(O).$$

- ▶ Therefore:

$$I(S; T(O)) \leq I(S; O).$$

Why this matters for analysing text

- ▶ NLP methods do not access meaning, intention, or social reality directly.
- ▶ They access an already encoded object:

phenomenon $\longrightarrow$ text $\longrightarrow$ model representation.

- ▶ This matters for every downstream transformation:

tokenisation, embedding, classification, summarisation, generation.

- ▶ These transformations may make information easier to use. They do not remove the original information constraint.

The practical lesson

Better models can improve extraction, compression, and prediction from available data. They cannot make the data contain what was never observed.

But isn't NLP just statistics over text?

- ▶ If NLP works with statistical models, why bring in communication at all?
- ▶ If text is just an observed sample, what exactly is the source upstream?
- ▶ If a model transforms text into embeddings, summaries, or predictions, how much information about the source can that transformation really add?
- ▶ And if the data are already encoded, what are we actually learning: the phenomenon itself, or only its textual trace?

The point

These are not separate questions. Statistical inference and communication are both information-processing problems. That is where the Data Processing Inequality becomes useful.

- In statistics, we observe data X and try to infer something upstream:

$$\theta \longrightarrow X \longrightarrow \hat{\theta}(X).$$

- This is the same canonical information structure as before:

$$S \longrightarrow O \longrightarrow T(O).$$

- Here, the source S is the unknown parameter θ , the observable O is the sample X , and the downstream transformation is the estimator $\hat{\theta}(X)$.
- Under this structure, the Data Processing Inequality applies directly:

$$I(\theta; \hat{\theta}(X)) \leq I(\theta; X).$$

Interpretation

Statistical inference cannot manufacture information about θ that was not already present in the data.

Fisher information is local

- ▶ Fisher information does not measure semantic content or global uncertainty. It measures how sensitive the likelihood is to small changes in a parameter.
- ▶ For a model $p(X | \theta)$, Fisher information is

$$\mathcal{I}(\theta) = \mathbb{E} \left[\left(\frac{\partial}{\partial \theta} \log p(X | \theta) \right)^2 \right].$$

- ▶ If nearby values of θ produce very similar distributions over X , then $\mathcal{I}(\theta)$ is small.
- ▶ If small changes in θ sharply change the likelihood, then $\mathcal{I}(\theta)$ is large.

Why it matters

Fisher information tells us how well the data can distinguish nearby models.

Fisher information is the local curvature of Shannon-type divergence

- ▶ Fisher information is tightly connected to the local geometry of Kullback–Leibler divergence.
- ▶ For a small perturbation $d\theta$,

$$D_{\text{KL}}(p(x | \theta) \| p(x | \theta + d\theta)) = \frac{1}{2} \mathcal{I}(\theta) d\theta^2 + o(d\theta^2).$$

- ▶ So Fisher information is a local Shannon-type quantity: it captures how fast distinguishability changes around θ .
- ▶ Under i.i.d. sampling,

$$\mathcal{I}_n(\theta) = n \mathcal{I}_1(\theta),$$

and precision improves only when genuinely new information enters through the data.

Takeaway

Fisher information is not extra information added by the estimator; it is the local informativeness of the likelihood itself.

Statistical inference is an information-theoretic process

- ▶ Statistical inference is not separate from information theory. It is an information-constrained attempt to extract something upstream from observed data.
- ▶ Let S denote the latent object of interest, O the observable data, and $T(O)$ any estimator or representation constructed from those data:

$$S \longrightarrow O \longrightarrow T(O).$$

- ▶ Under this canonical structure, the Data Processing Inequality gives:

$$I(S; T(O)) \leq I(S; O).$$

- ▶ So the basic point is simple: downstream processing cannot increase the information available about the source.

Interpretation

Inference is limited by the information structure of the data, not by the sophistication of the estimator alone.

- ▶ One useful way to think about modelling is as a source coding problem:

$$P \longrightarrow (X_1, \dots, X_n) \longrightarrow \hat{P}.$$

- ▶ Here, the true distribution P is the source, the sample $(X_1, \dots, X_n)$ is the observable data, and $\hat{P}$ is the compressed model built from those data.
- ▶ The model acts as an encoder: it tries to represent the data using fewer bits while preserving the relevant structure.
- ▶ This is why modelling is never just “fitting” in a mechanical sense. It is compression with constraints.

What the model can do

It can summarise and compress the data efficiently. It cannot recover information that was never in the sample.

Statistical inference as a noisy channel

- ▶ A second useful analogy is inference as a noisy channel problem:

$$f \longrightarrow (Y_1, \dots, Y_n) \longrightarrow \hat{f}.$$

- ▶ Here f is the latent function of interest, the observations are noisy outputs, and $\hat{f}$ is the decoder's estimate.
- ▶ In this picture, the goal is not to describe the data as compactly as possible. The goal is to recover the upstream source as accurately as possible.
- ▶ In both analogies, the message is the same: inference improves only when the data contain more independent information about the source.

Consequence

Whether we are modelling or inferring, the Data Processing Inequality remains the boundary condition.

10 min break

- ▶ Rich data sources retain a lot of the original phenomenon.
- ▶ They often carry multiple layers of information at once:

text, audio, image, video.

- ▶ But richness comes with complexity.
- ▶ They are usually harder to process, compare, and model directly.

Key point

More information is not always easier information.

Text is rich, but not transparent

- ▶ Text can preserve:

meaning, tone, style, context, ambiguity.

- ▶ This makes it especially valuable in social and health science.

- ▶ But it is also messy:

high-dimensional, context-dependent, variable.

- ▶ So text is informative, but not simple.

Trade-off

Text often contains more of the original communicative structure, but extracting it is harder.

Other sources are more structured

- ▶ Many datasets are much more structured:

tables, forms, records, coded variables.
- ▶ These sources are easier to analyse.
- ▶ But they are usually more compressed.
- ▶ Structure helps computation, but it often comes at the cost of detail.

Trade-off

Structured data is easier to handle, but often carries less of the original phenomenon.

Rich versus structured is an information trade-off

- ▶ Rich sources tend to preserve more information.
- ▶ Structured sources tend to be easier to use.
- ▶ The move from one to the other is not free:

phenomenon $\longrightarrow$ observable source $\longrightarrow$ representation.

- ▶ Every step can compress, simplify, or discard information.

Information-theoretic view

More structure usually means more usability, but also more loss.

The Data Processing Inequality gives the boundary

- ▶ Let S be the source phenomenon.
- ▶ Let O be the observed data.
- ▶ Let $T(O)$ be any downstream transformation.
- ▶ Then:

$$S \longrightarrow O \longrightarrow T(O)$$
$$I(S; T(O)) \leq I(S; O).$$

Interpretation

A more processed representation may be more convenient, but it cannot contain more information about the source than the source data already had.

Why this matters for text analysis

- ▶ Text is useful because it is rich.
- ▶ Text is hard because it is rich.
- ▶ NLP tries to make this rich source computationally tractable.
- ▶ But every representation is downstream of the original data:

text $\longrightarrow$ tokens $\longrightarrow$ embeddings $\longrightarrow$ model output.

Bottom line

The whole game is to extract as much as possible from the text without pretending the representation is the thing itself.

From words to meaning: the distributional move

- ▶ Classical NLP starts from a simple but powerful idea: words are not understood in isolation, but from the contexts in which they occur.
- ▶ This is the distributional hypothesis:

similar meanings tend to occur in similar contexts.

- ▶ Or, in Firth's phrasing:

you shall know a word by the company it keeps.

- ▶ This is the conceptual foundation of a large part of modern text analysis: count-based methods, embeddings, contextual representations, and eventually language models.

Basic intuition

Meaning is inferred from usage patterns, not read off from isolated symbols.

- ▶ Text is full of ambiguity. A word can mean different things depending on its surrounding context.
- ▶ For example:

They were sitting on the bank, watching the water rise.

The bank had already closed for the day.

- ▶ The same surface form appears in different contexts, but the surrounding words strongly constrain interpretation.
- ▶ The basic observation is that context carries information about meaning.

Historical point

Distributional semantics did not begin with neural networks. It began with the observation that linguistic usage itself contains statistical regularities.

Distributional semantics is not a noisy-channel model

- ▶ It is tempting to think of language analysis as a noisy-channel problem: latent meaning → noisy text → recovered meaning.
- ▶ But the distributional hypothesis does not need that assumption.
- ▶ It does not posit a hidden true meaning behind each use and then try to reconstruct it from corrupted observations.
- ▶ Instead, it treats usage itself as the object of analysis:

patterns of usage are the evidence.

Important distinction

Distributional systems do not decode meaning from noise. They compress regularities in observed usage.

A distributional system works by source coding

- ▶ The relevant structure is:

$$M \longrightarrow C \longrightarrow T(C),$$

where M is semantic content and C is observable linguistic realisation.

- ▶ The system sees only samples of C , not M directly.
- ▶ Its task is to build an internal representation $T(C)$ from those observed contexts.
- ▶ In this sense, a distributional system is closer to an encoder than a decoder.

Consequence

The representation is built from usage, and usage is all the system gets.

- ▶ The distributional hypothesis is the ancestor of word embeddings, contextual embeddings, and large language models.
- ▶ All of them rely on the same basic move: infer structure from co-occurrence, context, and statistical regularity.
- ▶ This makes them powerful, but also bounded: they operate entirely on observed text.
- ▶ So the information-theoretic question remains:

what is preserved in the representation, and what is lost?

Transition

This is where the next step begins: from co-occurrence to vectors, and from vectors to language models.

Tokenization: turning text into symbols the model can handle

- ▶ The first step is brutally simple: turn text into a finite vocabulary of tokens.
- ▶ Tokens are not just words; they can also include punctuation and other symbols.
- ▶ For example:

“I love my cat.” $\mapsto$ {I, love, my, cat, . }

- ▶ In practice, tokenization gives each token a numeric code.
- ▶ That is useful because neural models do not operate on raw text. They operate on numerical representations of text.

But the price is already visible

Tokenization is the first compression step: it discretises language into a smaller symbolic alphabet.

Bag-of-words: counting tokens, ignoring order

- ▶ A basic distributional method is to represent a document by token counts.
- ▶ This is the bag-of-words assumption:

what matters is which tokens appear, not their order.

- ▶ So a document becomes a vector of counts over the vocabulary.
- ▶ This is easy to compute, but it produces sparse matrices: most entries are zero most of the time.
- ▶ More importantly, it throws away sequence information, syntax, and a lot of context.

Information-theoretic reading

Bag-of-words is a lossy representation: it keeps frequency information, but discards much of the structure that made the text interpretable in the first place.

Why bag-of-words was eventually replaced

- ▶ Bag-of-words is useful, but it is a very rough compression of language.
- ▶ Two sentences with the same word counts can mean very different things:

“dog bites man” $\neq$ “man bites dog”

even though the bag-of-words representation is the same.

- ▶ So the representation is not injective: different texts can collapse onto the same vector.
- ▶ That means information is lost before the analysis even begins.

Why this matters

If the representation already destroys order and context, no downstream model can recover them without extra information.

`interactive tokenizer playground`

Word embeddings: the intuition

- ▶ The distributional idea is simple: words that appear in similar contexts tend to have similar meanings.
- ▶ So instead of treating a word as an arbitrary symbol, we map it to a vector:

$$w_i \mapsto \mathbf{v}_i \in \mathbb{R}^d.$$

- ▶ The embedding $\mathbf{v}_i$ places the word in a continuous space where proximity matters.
- ▶ Similar words end up close together; unrelated words end up far apart.
- ▶ This is the basic move behind modern distributional semantics.

Core idea

An embedding is a compressed representation of usage: it tries to keep the semantic regularities in the context while reducing everything else.

Word embeddings: a basic formalization (1/2)

- ▶ Let V be the vocabulary, with $|V|$ tokens. For each token $w_i \in V$, define its one-hot vector:

$$\mathbf{x}_i \in \mathbb{R}^{|V|}.$$

- ▶ The embedding layer is a matrix

$$E \in \mathbb{R}^{d \times |V|},$$

where $d \ll |V|$.

- ▶ The word vector for token w_i is obtained by lookup:

$$\mathbf{v}_i = E \mathbf{x}_i.$$

- ▶ In other words, the embedding of a word is just the column of E selected by its one-hot code.

Word embeddings: a basic formalization (2/2)

- ▶ A standard distributional objective is to predict contexts from target words.
- ▶ Let c denote a context token for a target word w . Then:

$$p(c \mid w) = \frac{\exp(\mathbf{v}_w^\top \mathbf{u}_c)}{\sum_{c' \in V} \exp(\mathbf{v}_w^\top \mathbf{u}_{c'})}.$$

- ▶ Here $\mathbf{u}_c$ is the context vector for token c .
- ▶ Training adjusts the vectors so that observed word-context pairs get high score:

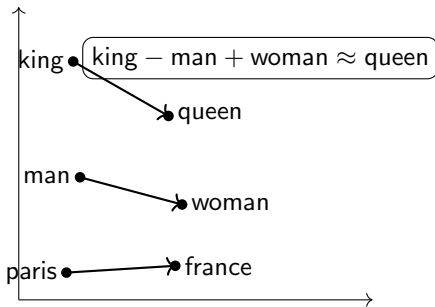
$$\mathbf{v}_w^\top \mathbf{u}_c \text{ large when } w \text{ and } c \text{ co-occur.}$$

- ▶ After training, we keep the word vectors $\{\mathbf{v}_i\}$ as the embeddings.

Interpretation

The model learns geometry from co-occurrence: nearby vectors correspond to similar distributions of usage.

Example: semantic structure in embedding space



- In embedding space, similarity becomes geometry.
- Words with related usage end up close together.
- Some linear relations also become visible.
- This is not magic; it is co-occurrence made numeric.

Takeaway

Word embeddings turn distributional information into a vector space that a model can manipulate.

- ▶ Once words are embedded as vectors, similarity is no longer based on exact string match. It is based on geometry.
- ▶ A standard measure is cosine similarity:

$$\cos(\mathbf{v}, \mathbf{u}) = \frac{\mathbf{v}^T \mathbf{u}}{\|\mathbf{v}\| \|\mathbf{u}\|}.$$

- ▶ This measures the angle between vectors, not their magnitude.
- ▶ So two embeddings can be considered similar if they point in roughly the same direction, even if their lengths differ.
- ▶ In practice, this is one of the main reasons embeddings are useful: they turn semantic proximity into a computable quantity.

TensorFlow Embedding Projector

- ▶ Static embeddings assign one vector to one word type.

$$w \mapsto \mathbf{v}_w$$

So bank gets the same vector whether it means a financial institution or a river bank.

- ▶ Dynamical or contextual embeddings relax this assumption: the vector depends on the surrounding context.

$$w \mapsto \mathbf{v}(w \mid \text{context}).$$

- ▶ So the same token can live in different regions of the space depending on the sentence it appears in.
- ▶ This is a major shift: meaning is no longer attached only to the word type, but to the word in use.
- ▶ That is exactly why modern models can distinguish between:

“bank loan” and “river bank”.

Embeddings approximate meaning, but not structure

- ▶ Word embeddings gave us a dense approximation of meaning:

$$w \mapsto \mathbf{v}_w \in \mathbb{R}^d.$$

- ▶ But a static embedding is still a baggy object: it says something about similarity, but very little about sequence.
- ▶ In particular, it does not by itself encode:

order, distance, hierarchy, long-range dependence.

- ▶ That is the next problem NLP had to solve: how to keep the semantic advantages of embeddings while recovering the structure of text.

Transition

Embeddings compress meaning; modern sequence models add structure back in.

Positional encoding puts order back into the representation

- ▶ Transformers do not process text as an ordered recurrence. They process it as a set of token vectors, so the model must be told where each token sits.
- ▶ If $\mathbf{e}_i$ is the embedding of token i , the input becomes:

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i,$$

where $\mathbf{p}_i$ is the positional encoding.

- ▶ A standard sinusoidal encoding is:

$$p_{pos,2k} = \sin\left(\frac{pos}{10000^{2k/d_{\text{model}}}}\right), \quad p_{pos,2k+1} = \cos\left(\frac{pos}{10000^{2k/d_{\text{model}}}}\right).$$

- ▶ The point is not cosmetic. It is structural: the model now sees both *what* a token is and *where* it appears.

Interpretation

Positional encoding turns a bag of vectors into a sequence.

Self-attention lets each token look at the others

- ▶ The core transformer operation is self-attention. For each token, the model computes three projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

- ▶ Attention scores are computed by comparing queries and keys:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- ▶ The softmax turns similarity scores into weights. Each token receives a weighted mixture of the value vectors of all the other tokens.
- ▶ So the representation of a token becomes context-dependent: it is updated according to the tokens it attends to.

What this buys us

Attention gives the model a direct way to model long-range dependence without having to pass information step by step through a recurrent chain.

Multi-head attention and transformer blocks

- ▶ One attention pattern is useful. Several in parallel are better.
- ▶ Multi-head attention computes multiple attention maps:

$$\text{head}_j = \text{Attention}(XW_Q^{(j)}, XW_K^{(j)}, XW_V^{(j)}).$$

- ▶ The heads are concatenated and projected again:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

- ▶ In the full transformer block, attention is combined with a feed-forward layer, residual connections, and layer normalisation.
- ▶ The result is a model that is both expressive and trainable at scale.

Why this matters

Different heads can learn different relations: syntax, coreference, topic continuity, and other dependencies.

- ▶ Before transformers, sequence-to-sequence tasks were dominated by RNNs and LSTMs.
- ▶ They worked, but they were awkward: sequential computation, harder optimisation, and weak long-range memory.
- ▶ Transformers changed the game by using embeddings, positional encodings, and attention instead of recurrence.
- ▶ This made them easier to parallelise and much better at learning from large corpora.
- ▶ The architecture became the basis of modern machine translation, text classification, summarisation, question answering, and text generation.

Where this ends up

Decoder-only transformers, trained for next-token prediction, are the backbone of modern chatbots and large language models.

- ▶ The basic training objective of modern language models is not to classify a whole text upfront. It is to predict the next token from the previous ones:

$$p(x_t \mid x_{<t}).$$

- ▶ Given a sequence $x_1, \dots, x_T$, the model is trained to maximise:

$$\sum_{t=1}^T \log p(x_t \mid x_{<t}).$$

- ▶ In other words, the model learns a conditional distribution over the next token at each step.
- ▶ This is why the architecture needs a causal mask: token t is only allowed to attend to tokens before it, not after it.
- ▶ So the model is not “reading the whole answer in advance”. It is learning to continue text one token at a time.

A useful interactive visualisation

- ▶ If you want one visual that makes the transformer architecture feel less abstract, this is a good one.
- ▶ It shows the flow from tokens to embeddings, attention, and next-token prediction.
- ▶ It is useful as a companion reference when we move into the next lecture.

Interactive demo

`bbycroft.net/llm`